

Lab4：地址空间

本章完成的工作

本章完成了地址空间与虚拟内存机制的理解和编程作业：

- 1. 成功运行 ch4 代码，观察虚拟内存机制下的程序执行
- 2. 理解动态内存分配：kalloc/kfree 物理页帧分配器
- 3. 理解地址空间抽象和虚实地址转换
- 4. 深入理解 SV39 多级页表机制
- 5. 理解页表操作：walk、mappages、uvmunmap
- 6. 理解跨页表数据传递：copyin/copyout
- 7. 实现 sys_mmap 和 sys_munmap 系统调用（编程作业）
- 8. 修复 sys_gettimeofday 和 sys_trace 以适应虚拟内存

本章有编程作业：实现 mmap/munmap 系统调用，并修复第三章的系统调用以支持虚拟内存。

报告结构说明

本报告的结构安排及与清华指导书的对比：

清华指导书小节	本报告对应小节	差异说明
本章导读（引言）	第二节：从物理内存到虚拟内存	从 Lab3 的问题出发，理解虚拟内存的必要性
C 的动态内存分配	第三节：动态内存分配	解释 kalloc/kfree 的链表实现
地址空间	第四节：地址空间抽象	增加地址空间演进历史（分段→分页）
SV39多级页表（内容介绍）	第五节：SV39 多级页表原理	增加 PTE 结构图和地址转换示例
SV39多级页表（OS实现）	第六节：页表操作函数分析	分析 walk/mappages/copyin/copyout
chapter4练习	第七节：编程作业 mmap/munmap	记录 freewalk panic 问题的排查过程

本报告的特点：

- 1. 不是照搬指导书：每个概念都用自己的话解释，体现"不懂 → 探索 → 理解"的过程
- 2. 增加可视化内容：SV39 地址结构图、页表层次图、地址转换流程图
- 3. 记录真实问题：freewalk: leaf panic、NULL 未定义等排查过程
- 4. 前后呼应：从 Lab3 的直接访问物理地址引出本章的问题，形成知识串联

一、实验环境与运行

1.1 代码目录结构

2025-ucore-riscv-清华/
├─ uCore-Tutorial-Code-2025S-ch4/
│ ├── os/
│ │ ├── syscall.c
│ │ ├── vm.c
│ │ └── ...
│ └─ user/
│ └─ src/
│ ├── ch4_mmap0.c
│ ├── ch4_unmap0.c
│ └── ...
└─ os-btbu/

← 本章代码

← 内核代码（需要修改）

← 添加 mmap/munmap

← 修改 freewalk

← 用户测试程序

← mmap 基础测试

← munmap 测试

← 最终完成的代码

1.2 本章新增/修改的关键文件

相比 Lab3，本章内核代码有明显变化：

文件	新增/修改	说明
os/kalloc.c	新增	物理页帧分配器
os/kalloc.h	新增	分配器头文件
os/vm.c	新增	虚拟内存管理
os/vm.h	新增	虚拟内存头文件

1.3 运行命令与结果

```
cd /桌面/herdream/2025-ucore-riscv-清华/uCore-Tutorial-Code-2025S-ch4（其他同学可根据实际路径）  
make clean  
make user CHAPTER=4  
make run
```

运行结果（关键部分）：

Test 04_0 OK!
Test 04_4 ummap OK!
Test 04_5 ummap2 OK!
Test 04_3 test OK!
Test trace_1 OK!
Test trace OK!
Test sbrk almost OK!

← mmap 基本功能测试通过

← munmap 测试通过

← munmap 测试2通过

← mmap 综合测试通过

← trace 在虚拟内存下测试通过

← trace 完整测试通过

← sbrk 测试通过

二、从物理内存到虚拟内存：引言的理解

2.1 回顾 Lab3：直接使用物理地址的问题

在 Lab3 中，我们的程序直接使用物理地址访问内存。当时觉得这样很直接、很简单。

但仔细想想，指导书提到的三个问题确实很严重：

1. **对应用开发者不友好**：程序员需要知道自己的程序会被加载到哪个物理地址，不同程序之间还要协调避免冲突。
2. **没有内存保护**：任何程序都可以访问整个物理内存，包括其他程序的数据和内核代码。一个 bug 或恶意程序可以搞崩整个系统。
3. **内存利用不灵活**：程序的内存空间在运行前就固定了，运行结束后释放的空间也不能动态分配给其他程序。

2.2 虚拟内存的核心思想

虚拟内存的解决方案很优雅：**给每个程序一个假象，让它以为自己独占整个内存空间。**

程序使用的地址不再是真实的物理地址，而是**虚拟地址**。操作系统和硬件配合，在程序访问内存时自动把虚拟地址**翻译**成物理地址。

程序视角：

我有一块从 0 开始的连续内存空间，可以随意使用

实际情况：

程序的数据分散在物理内存的各个位置
每次访问都要通过页表转换地址

2.3 硬件支持：MMU 和页表

虚拟地址到物理地址的转换不能完全由软件完成（太慢了）。RISC-V 提供了硬件支持：

- **MMU（内存管理单元）**：自动进行地址转换
- **TLB（转址旁路缓存）**：缓存最近使用的地址映射，加速转换
- **satp 寄存器**：告诉 MMU 使用哪个页表

2.4 本章的目标

理解了背景后，本章的目标就清晰了：

1. **实现物理内存管理**：kalloc/kfree 分配和释放物理页帧
2. **理解 SV39 页表机制**：RISC-V 的三级页表结构
3. **实现虚拟内存操作**：mappages、uvmunmap、walk 等函数
4. **实现 mmap/munmap**：让用户程序可以动态申请和释放虚拟内存

三、动态内存分配

3.1 物理内存的范围

首先需要知道我们有多少物理内存可用。在 `memory_layout.h` 中定义：

```
#define KERNBASE 0x80200000L
#define PHYSTOP (0x80000000 + 128*1024*1024) // 128M
```

物理内存范围是 `[0x80000000, 0x88000000)`，共 128MB。但其中 `[0x80000000, 0x80200000)` 被 RustSBI 占用，内核从 `0x80200000` 开始。

3.2 kalloc/kfree：链表式分配器

指导书介绍了用链表管理空闲物理页帧的方法。我仔细阅读了 `kalloc.c` 的代码：

```
// os/kalloc.c

struct linklist {
    struct linklist *next;
};

struct {
    struct linklist *freelist; // 指向空闲页链表的头
} kmem;
```

关键洞察：空闲页帧本身就是链表节点！

一个空闲页帧有 4KB，我们只用前 8 个字节存放指向下一个空闲页帧的指针，其余空间暂时不用。这样就不需要额外的内存来维护链表结构。

空闲页帧（4KB）：

next 指针（8字节）	未使用（4088字节）
--------------	-------------

3.3 分配过程（kalloc）

```
void *kalloc(void)
{
    struct linklist *l;
    l = kmem.freelist; // 取链表头
    if (l) {
        kmem.freelist = l->next; // 更新链表头
        memset((char *)l, 5, PGSIZE); // 填充垃圾值，便于调试
    }
    return (void *)l;
}
```

分配就是从链表头摘下一个节点，时间复杂度 $O(1)$ 。

3.4 释放过程 (kfree)

```
void kfree(void *pa)
{
    // 安全检查
    if (((uint64)pa % PGSIZE) != 0 ||
        (char *)pa < ekernel ||
        (uint64)pa >= PHYSTOP) {
        panic("kfree");
    }

    memset(pa, 1, PGSIZE); // 填充垃圾值

    // 插入链表头
    struct linklist *l = (struct linklist *)pa;
    l->next = kmem.freelist;
    kmem.freelist = l;
}
```

释放就是把页帧插入链表头，也是 O(1)。

3.5 初始化 (kinit)

在系统启动时，把所有可用物理页帧都加入空闲链表：

```
void kinit()
{
    freerange(ekernel, (char *)PHYSTOP);
}

void freerange(char *pa_start, char *pa_end)
{
    char *p = (char *)PGROUNDUP((uint64)pa_start);
    for (; p + PGSIZE <= pa_end; p += PGSIZE) {
        kfree(p); // 把每一页都"释放"到链表中
    }
}
```

四、地址空间抽象

4.1 地址空间的演进

指导书介绍了地址空间抽象的演进历史，我整理如下：

阶段	方案	优点	缺点
裸机	程序直接使用物理地址	简单	只能运行一个程序
批处理	程序顺序执行，共用物理内存	可以运行多个程序	同时只有一个程序在内存中

阶段	方案	优点	缺点
多道程序	多个程序同时在内存中，每个占据固定区域	切换快	需要预先规划地址，没有保护
分段	每个逻辑段有独立的 base/bound	灵活	外部碎片问题
分页	以固定大小的页为单位管理	无外部碎片，管理简单	需要页表，有少量内部碎片

4.2 分段 vs 分页

分段的问题：

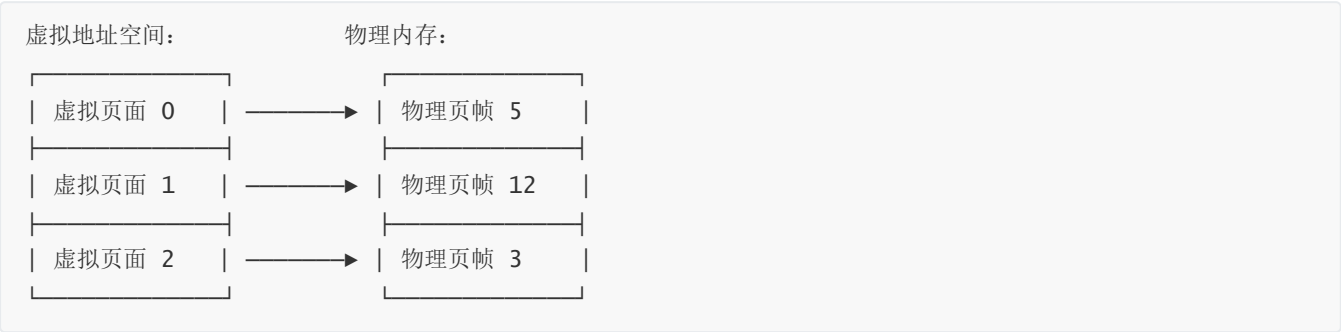
- 每个段的大小不同，内存分配和回收后会产生"外部碎片"
- 外部碎片需要"内存紧缩"来整理，开销很大

分页的优势：

- 以固定大小（4KB）的页为单位分配
- 不会产生外部碎片
- 内存管理可以用简单的位图或链表

4.3 虚拟页面和物理页帧

分页机制下，虚拟地址空间被划分为**虚拟页面**（Page），物理内存被划分为**物理页帧**（Frame），大小都是 4KB。



虚拟页面到物理页帧的映射关系存储在**页表**中。

五、SV39 多级页表原理

5.1 为什么需要多级页表？

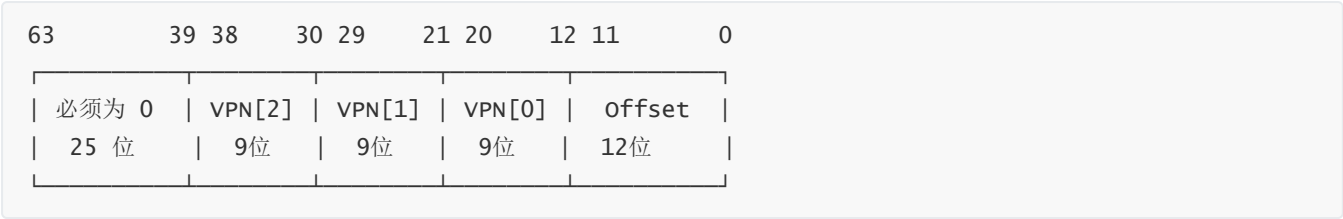
如果用简单的线性表存储页表，每个虚拟页面一个条目：

- 27 位虚拟页号 → $2^{27} = 128\text{M}$ 个条目
- 每个条目 8 字节 → 页表大小 = 1GB！

这显然不可行。解决方案是**多级页表**：只为实际使用的地址范围分配页表空间。

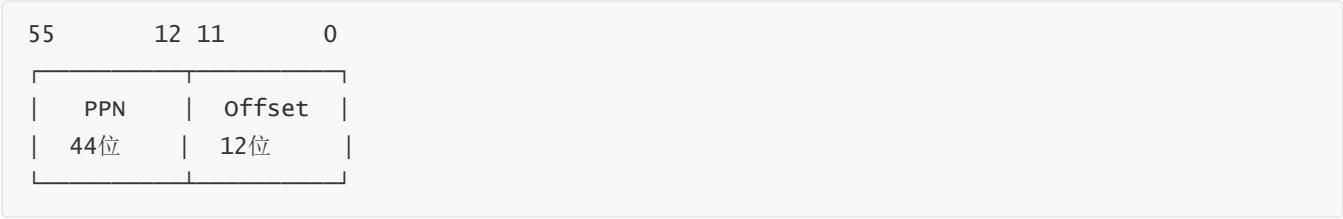
5.2 SV39 地址格式

SV39 的名字来自于 39 位虚拟地址：



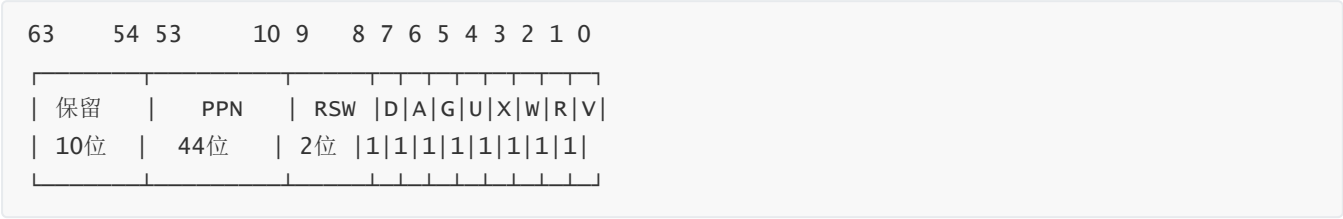
- **VPN[2], VPN[1], VPN[0]**: 三级页表索引，每级 9 位
- **Offset**: 页内偏移，12 位（对应 4KB 页大小）

物理地址是 56 位：



5.3 页表条目（PTE）格式

每个页表条目是 64 位：

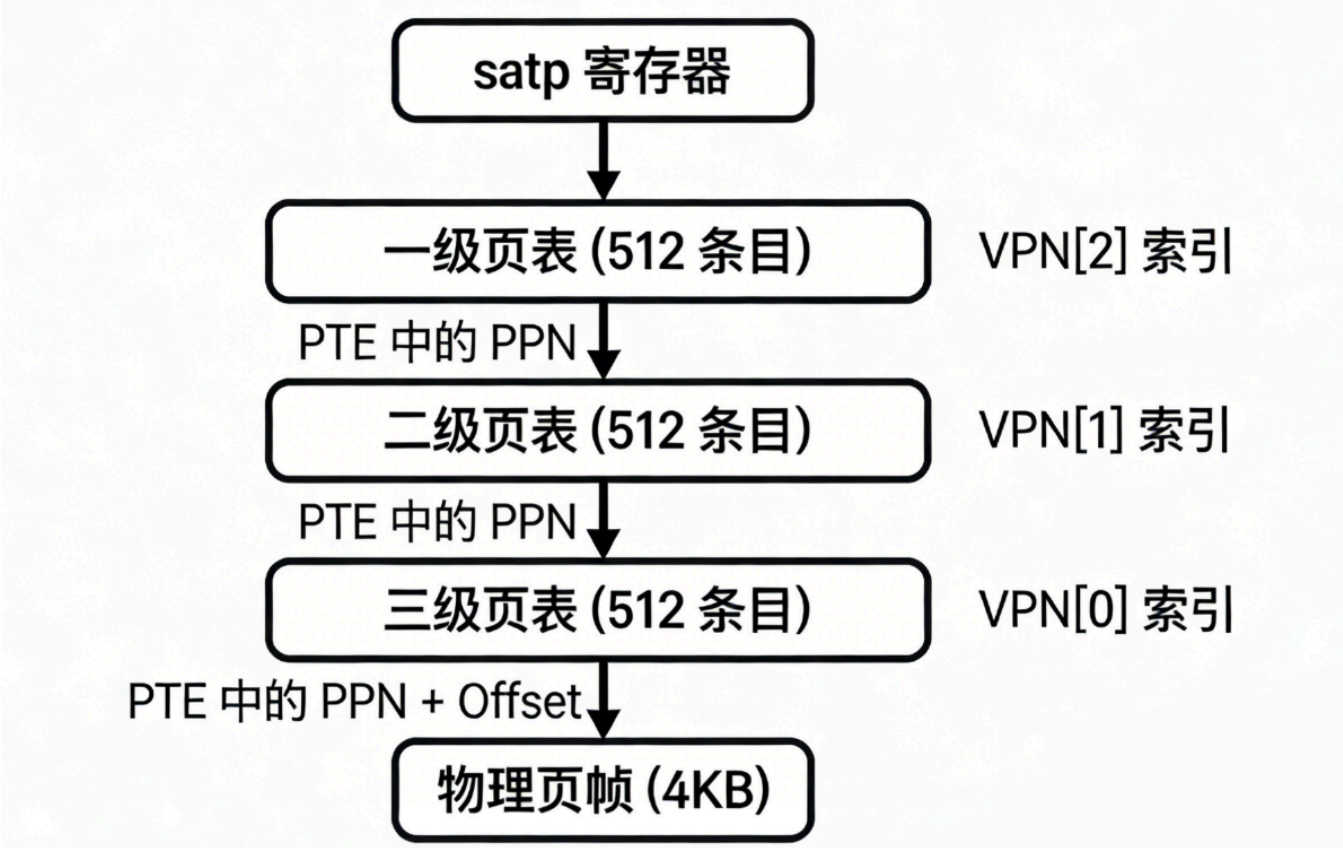


关键标志位：

标志	含义
V	Valid，条目是否有效
R	Read，可读
W	Write，可写
X	Execute，可执行
U	User，用户态可访问

5.4 三级页表结构

【原创结构图1：SV39 三级页表结构】



每级页表有 512 个条目 (2^9)，每个条目 8 字节，所以每个页表正好占用一个 4KB 页帧。

5.5 地址转换过程

给定虚拟地址 VA，转换步骤：

1. 从 satp 寄存器获取一级页表的物理地址
2. 用 VPN[2] 作为索引，找到一级 PTE
3. 从一级 PTE 获取二级页表的物理地址
4. 用 VPN[1] 作为索引，找到二级 PTE
5. 从二级 PTE 获取三级页表的物理地址
6. 用 VPN[0] 作为索引，找到三级 PTE
7. 从三级 PTE 获取物理页帧地址
8. 物理页帧地址 + Offset = 最终物理地址

5.6 多级页表节省内存的原理

多级页表为什么能节省内存？

关键在于只分配实际使用的页表。

假设一个程序只使用了 1MB 的虚拟地址空间（256 个页面）：

- 线性表：需要 2^{27} 个条目 = 1GB

- 多级页表：
 - 一级页表：1 个 (4KB)
 - 二级页表：1 个 (4KB)
 - 三级页表：1 个 (4KB)
 - 总共只需要 12KB!

六、页表操作函数分析

6.1 walk: 查找 PTE

`walk` 函数是页表操作的核心，根据虚拟地址找到对应的 PTE：

```
pte_t *walk(pagetable_t pagetable, uint64 va, int alloc)
{
    if (va >= MAXVA)
        panic("walk");

    for (int level = 2; level > 0; level--) {
        pte_t *pte = &pagetable[PX(level, va)]; // 取对应级别的索引
        if (*pte & PTE_V) {
            pagetable = (pagetable_t)PTE2PA(*pte); // 进入下一级页表
        } else {
            if (!alloc || (pagetable = (pde_t *)kalloc()) == 0)
                return 0;
            memset(pagetable, 0, PGSIZE);
            *pte = PA2PTE(pagetable) | PTE_V; // 创建新的页表页
        }
    }
    return &pagetable[PX(0, va)]; // 返回最终的 PTE
}
```

`alloc` 参数控制是否在页表不存在时创建：

- `alloc=1`：创建中间页表（用于 mappages）
- `alloc=0`：不创建，找不到就返回 0（用于检查地址是否已映射）

6.2 mappages: 建立映射

```
int mappages(pagetable_t pagetable, uint64 va, uint64 size, uint64 pa, int perm)
{
    uint64 a = PGROUNDDOWN(va);
    uint64 last = PGROUNDDOWN(va + size - 1);

    for (;;) {
        pte_t *pte = walk(pagetable, a, 1); // 找到或创建 PTE
        if (pte == 0)
            return -1;
        if (*pte & PTE_V) // 已经映射
```

```

        return -1;
    *pte = PA2PTE(pa) | perm | PTE_V; // 设置映射
    if (a == last)
        break;
    a += PGSIZE;
    pa += PGSIZE;
}
return 0;
}

```

6.3 uvmunmap: 取消映射

```

void uvmunmap(pagetable_t pagetable, uint64 va, uint64 npages, int do_free)
{
    for (uint64 a = va; a < va + npages * PGSIZE; a += PGSIZE) {
        pte_t *pte = walk(pagetable, a, 0);
        if (pte == 0)
            continue;
        if ((*pte & PTE_V) != 0) {
            if (do_free) {
                uint64 pa = PTE2PA(*pte);
                kfree((void *)pa); // 释放物理页帧
            }
        }
        *pte = 0; // 清除 PTE
    }
}

```

6.4 copyin/copyout: 跨页表数据传递

用户程序的数据在用户页表中，内核无法直接访问用户的虚拟地址。需要通过 `copyin/copyout` 进行跨页表的数据拷贝。

```

// 从用户空间复制到内核
int copyin(pagetable_t pagetable, char *dst, uint64 srcva, uint64 len)
{
    while (len > 0) {
        uint64 va0 = PGROUNDDOWN(srcva);
        uint64 pa0 = walkaddr(pagetable, va0); // 转换用户虚拟地址
        if (pa0 == 0)
            return -1;
        uint64 n = PGSIZE - (srcva - va0);
        if (n > len)
            n = len;
        memmove(dst, (void *) (pa0 + (srcva - va0)), n);
        len -= n;
        dst += n;
        srcva = va0 + PGSIZE;
    }
    return 0;
}

```

这就是为什么 Lab4 需要修复 `sys_gettimeofday`：Lab3 直接使用用户传入的指针，在启用虚拟内存后不再有效，需要用 `copyout` 来写入。

七、编程作业：mmap/munmap

7.1 任务描述

实现两个内存映射相关的系统调用：

系统调用	调用号	功能
<code>sys_mmap</code>	222	将一段虚拟地址空间映射到物理内存
<code>sys_munmap</code>	215	取消已有的内存映射

`sys_mmap` 接口：

```
int sys_mmap(uint64 start, uint64 len, int prot, int flags);
// start: 起始虚拟地址，必须页对齐
// len: 映射长度
// prot: 权限位 (bit0=读, bit1=写, bit2=执行)
// flags: 忽略
// 返回值: 成功返回 0, 失败返回 -1
```

`sys_munmap` 接口：

```
int sys_munmap(uint64 start, uint64 len);
// start: 起始虚拟地址，必须页对齐
// len: 取消映射的长度
// 返回值: 成功返回 0, 失败返回 -1
```

7.2 mmap 的本质

理解 mmap 在做什么对实现很重要：

mmap 的本质是建立虚拟地址到物理地址的映射关系。

用户程序请求："我想使用从地址 X 开始的 N 个字节的内存"

内核需要做的：

- 1. 检查这段地址是否可用（没有被其他映射占用）
- 2. 分配物理页面
- 3. 在页表中建立映射

7.3 权限位转换

mmap 的 `prot` 参数格式与 PTE 的权限位格式不同，需要转换：

prot 位	含义	PTE 权限位
bit 0	可读	PTE_R (bit 1)
bit 1	可写	PTE_W (bit 2)
bit 2	可执行	PTE_X (bit 3)

```
int perm = PTE_U; /* 用户可访问 */
if (prot & 0x1) perm |= PTE_R;
if (prot & 0x2) perm |= PTE_W;
if (prot & 0x4) perm |= PTE_X;
```

7.4 sys_mmap 实现

```
/* ch4: sys_mmap - 匿名内存映射 */
int sys_mmap(uint64 start, uint64 len, int prot, int flags)
{
    struct proc *p = curr_proc();

    /* ch4: 检查起始地址页对齐 */
    if (!PGALIGNED(start))
        return -1;

    /* ch4: 检查prot有效性: 其他位必须为0 */
    if ((prot & ~0x7) != 0)
        return -1;

    /* ch4: 检查prot至少有一个权限(R/W/X) */
    if ((prot & 0x7) == 0)
        return -1;

    /* ch4: len为0时直接返回成功 */
    if (len == 0)
        return 0;

    /* ch4: 将len向上取整到页边界 */
    uint64 npages = (len + PGSIZE - 1) / PGSIZE;

    /* ch4: 检查[start, start+len)是否已被映射 */
    for (uint64 va = start; va < start + npages * PGSIZE; va += PGSIZE) {
        pte_t *pte = walk(p->pagetable, va, 0);
        if (pte != 0 && (*pte & PTE_V) != 0)
            return -1; /* 已被映射 */
    }

    /* ch4: 将prot转换为PTE标志位 */
```

```

int perm = PTE_U;
if (prot & 0x1) perm |= PTE_R;
if (prot & 0x2) perm |= PTE_W;
if (prot & 0x4) perm |= PTE_X;

/* ch4: 逐页映射 */
for (uint64 i = 0; i < npages; i++) {
    uint64 va = start + i * PGSIZE;
    char *mem = kalloc();
    if (mem == 0)
        return -1;
    memset(mem, 0, PGSIZE);
    if (mappages(p->pagetable, va, PGSIZE, (uint64)mem, perm) != 0) {
        kfree(mem);
        return -1;
    }
}

return 0;
}

```

7.5 sys_munmap 实现

```

/* ch4: sys_munmap - 取消内存映射 */
int sys_munmap(uint64 start, uint64 len)
{
    struct proc *p = curr_proc();

    /* ch4: 检查起始地址页对齐 */
    if (!PGALIGNED(start))
        return -1;

    /* ch4: len为0时直接返回成功 */
    if (len == 0)
        return 0;

    /* ch4: 将len向上取整到页边界 */
    uint64 npages = (len + PGSIZE - 1) / PGSIZE;

    /* ch4: 检查[start, start+len)是否全部已映射 */
    for (uint64 va = start; va < start + npages * PGSIZE; va += PGSIZE) {
        pte_t *pte = walk(p->pagetable, va, 0);
        if (pte == 0 || (*pte & PTE_V) == 0)
            return -1; /* 未映射 */
    }

    /* ch4: 取消映射并释放页面 */
    uvmunmap(p->pagetable, start, npages, 1);

    return 0;
}

```

7.6 修复 sys_gettimeofday

Lab3 的实现直接使用用户指针，在虚拟内存下会出问题：

```
// Lab3 的实现（有问题）
uint64 sys_gettimeofday(TimeVal *val, int _tz)
{
    val->sec = ...;    // 直接写入用户地址，在虚拟内存下无效！
    val->usec = ...;
    return 0;
}
```

修复后使用 `copyout`：

```
/* ch4: 修复sys_gettimeofday, 使用copyout处理用户指针 */
uint64 sys_gettimeofday(uint64 val, int _tz)
{
    struct proc *p = curr_proc();
    uint64 cycle = get_cycle();
    TimeVal t;
    t.sec = cycle / CPU_FREQ;
    t.usec = (cycle % CPU_FREQ) * 1000000 / CPU_FREQ;
    copyout(p->pagetable, val, (char *)&t, sizeof(TimeVal));
    return 0;
}
```

7.7 修复 sys_trace 权限检查

mmap 可以创建只读或只写的页面，`sys_trace` 需要检查权限：

```
/* ch4: 更新以检查虚存读写权限 */
int sys_trace(int trace_request, uint64 id, uint8 data)
{
    struct proc *p = curr_proc();
    if (trace_request == 0) {
        /* ch4: 读操作 - 检查地址是否用户可见且可读 */
        pte_t *pte = walk(p->pagetable, id, 0);
        if (pte == 0 || (*pte & PTE_V) == 0 ||
            (*pte & PTE_U) == 0 || (*pte & PTE_R) == 0)
            return -1;
        uint8 *addr = (uint8 *)useraddr(p->pagetable, id);
        if (addr == 0)
            return -1;
        return *addr;
    } else if (trace_request == 1) {
        /* ch4: 写操作 - 检查地址是否用户可见且可写 */
        pte_t *pte = walk(p->pagetable, id, 0);
        if (pte == 0 || (*pte & PTE_V) == 0 ||
            (*pte & PTE_U) == 0 || (*pte & PTE_W) == 0)
            return -1;
        // ...
    }
}
```

```
}  
// ...  
}
```

7.8 遇到的问题与解决

问题1: 编译错误 'NULL' undeclared

原因: ch4 的内核代码没有包含定义 NULL 的头文件。

解决: 将 NULL 改为 0。在 C 语言中，空指针可以用 0 或 NULL 表示，效果相同。

问题2: 进程退出时 freewalk: leaf panic

现象: 运行 mmap 测试时，进程退出后出现 freewalk: leaf panic。

分析:

1. mmap 可以在任意地址分配内存（只要没被占用）
2. 进程的 max_page 字段只记录程序代码和栈的范围
3. uvmfree 只清理 0 到 max_page 范围的页面
4. mmap 分配的高地址页面没有被清理，导致 freewalk 遇到"意外"的叶子页面

解决: 修改 freewalk 函数，遇到叶子页面时释放它而不是 panic:

```
/* ch4: 修改为同时释放叶子页面，支持mmap区域的清理 */  
void freewalk(pagetable_t pagetable)  
{  
    for (int i = 0; i < 512; i++) {  
        pte_t pte = pagetable[i];  
        if ((pte & PTE_V) && (pte & (PTE_R | PTE_W | PTE_X)) == 0) {  
            // 指向下级页表  
            uint64 child = PTE2PA(pte);  
            freewalk((pagetable_t)child);  
            pagetable[i] = 0;  
        } else if (pte & PTE_V) {  
            /* ch4: 释放叶子页面的物理内存 */  
            uint64 pa = PTE2PA(pte);  
            kfree((void *)pa);  
            pagetable[i] = 0;  
        }  
    }  
    kfree((void *)pagetable);  
}
```

7.9 测试结果

Test 04_0 OK!	← mmap 基本功能
Test 04_3 test OK!	← mmap 综合测试
Test 04_4 ummap OK!	← munmap 测试
Test 04_5 ummap2 OK!	← munmap 测试2
Test trace_1 OK!	← trace 权限检查
Test trace OK!	← trace 完整测试

八、本章新增系统调用汇总

系统调用	调用号	功能	来源
sys_write	64	输出字符串	Lab2 已有
sys_exit	93	退出进程	Lab2 已有
sys_sched_yield	124	主动让出 CPU	Lab3 已有
sys_gettimeofday	169	获取当前时间	Lab3 已有，本章修复
sys_sbrk	214	调整堆大小	框架提供
sys_munmap	215	取消内存映射	本章作业
sys_mmap	222	内存映射	本章作业
sys_trace	410	追踪系统调用	Lab3 已有，本章修复

九、实验总结

完成情况

- ☑ 理解动态内存分配 (kalloc/kfree)
- ☑ 理解地址空间抽象和虚实地址转换
- ☑ 理解 SV39 多级页表机制
- ☑ 理解 PTE 结构和权限位
- ☑ 理解页表操作函数 (walk/mappages/uvmunmap)
- ☑ 理解跨页表数据传递 (copyin/copyout)
- ☑ 实现 sys_mmap 系统调用
- ☑ 实现 sys_munmap 系统调用
- ☑ 修复 sys_gettimeofday 使用 copyout
- ☑ 修复 sys_trace 权限检查
- ☑ 修改 freewalk 支持 mmap 区域清理
- ☑ 通过所有测试

收获与体会

- 1. **虚拟内存是操作系统最重要的抽象之一**：它解决了内存保护、地址冲突、动态分配等一系列问题。
- 2. **多级页表的设计很精妙**：通过树形结构，只为实际使用的地址分配页表空间，大大节省了内存。
- 3. **理解"为什么"比"怎么做"更重要**：比如 freewalk 的问题，如果只知道"怎么修复"而不理解"为什么会 panic"，下次遇到类似问题还是会卡住。
- 4. **内核和用户态的界限更加清晰**：在启用虚拟内存后，内核不能直接访问用户地址，必须通过 copyin/copyout。这种隔离是安全性的基础。
- 5. **系统调用接口的连续性**：Lab3 的 sys_trace 在 Lab4 需要修改，说明系统调用的实现不是一成不变的，需要随着内核功能的演进而调整。

文件修改清单

文件	修改内容
os/syscall_ids.h	添加 SYS_trace 410
os/proc.h	添加 MAX_SYSCALL_NUM 和 syscall_count[] 数组
os/vm.h	添加 walk() 函数声明
os/vm.c	修改 freewalk() 支持 mmap 区域清理
os/syscall.c	实现 sys_mmap()、sys_munmap()，修复 sys_gettimeofday()、sys_trace()

十、验证截图

```
hjl@hjl-VMware-Virtual-Platform: ~/桌面/herdream/2025-ucore-riscv-清华/uCore-Tutorial-Code-2025S-ch4
Test 04_5 ummap2 OK!
AAAAAAAAAA [2/5]
CCCCCCCCCC [2/5]
BBBBBBBBBB [2/5]
Test 04_3 test OK!
AAAAAAAAAA [3/5]
CCCCCCCCCC [3/5]
BBBBBBBBBB [3/5]
AAAAAAAAAA [4/5]
CCCCCCCCCC [4/5]
BBBBBBBBBB [4/5]
AAAAAAAAAA [5/5]
CCCCCCCCCC [5/5]
BBBBBBBBBB [5/5]
Test write A OK!
Test write C OK!
Test write B OK!
time_msec = 282 after sleeping 100 ticks, delta = 100ms!
Test sleep1 passed!
string from task trace test
Test trace OK!
Test sleep OK!
hjl@hjl-VMware-Virtual-Platform: ~/桌面/herdream/2025-ucore-riscv-清华/uCore-Tutorial-Code-2025S-ch4$
```

关键输出：

```
Test 04_0 OK!  
Test 04_3 test OK!  
Test 04_4 ummap OK!  
Test 04_5 ummap2 OK!  
Test trace_1 OK!  
Test trace OK!
```

所有 ch4 相关测试通过，说明 mmap/munmap 实现正确。